Cloud Computing Lab 4: Kubernetes

Due Date

Monday, October 13, 2025 11:59 PM

Objectives

This lab will focus on Kubernetes and how to run your Docker image through it locally, using YAML files as our mode of transportation.

What is Kubernetes?

Kubernetes (K8s) is an open-source platform that allows users to maintain, manage, and update their dockerized applications. There are plenty of different services out there from the big tech companies that offer even more streamlined processes to use K8s (like Google's GKE, but that's spoilers for Lab 5). For now, we're going to do these things locally, which requires some setup...

The Setup

Now, we'll need to install `minikube`, which is a service that creates a K8s cluster for you to work in. Follow these steps and you should be good. Note that I believe `minikube` requires a decent amount of storage to run, and it will tell you if you don't meet that requirement. If you don't, feel free to use a VM from WashU or find a different computer to use.

YAML Files

The only code we'll be writing for this lab is configuration for YAML files. You should have seen an example or two in class of YAML files, so I'm not going to go into depth about them. Instead, I'm going to detail what specific configurations you'll need and link you to a few example of them. For this lab, you'll need:

- Deployment - Jump to YAML
- Service - Jump to YAML
- ConfigMap - Jump to YAML
- Secret - Jump to YAML

With these examples, you should be able to customize your own copies to use your own image and "apply" (hmm... that sounds like a CLI command...) them to your cluster. If there's anything not covered in this doc, there are plenty of examples and documentations out there for K8s YAML files.

DIY

For this lab, you're just going to video yourself with a cluster running your docker image, and what you will turn in will be this video. There should be 3 replicates, no AWS keys exposed anywhere, and able to be viewed online. You can expose your app to run on the web (locally) by running:

```
> minikube service <name of service>
```

You should see a link to http://127.0.0.1:<port>, which you can click on, and your API will go from there! Then create a video that displays your movie data. This video should be accessible to the TAs (either commit the video to GitHub or provide a link to Box or Google drive). Furthermore, we want to be sure your cluster has the capacity to delete a pod and it automatically reconstructs itself. So, demo in your video listing the three pods, deleting one, then listing them again to show it's a new pod being set up. This is an example of the self healing element of kubernetes!!

Grading

- 20 - Minikube is running docker image
- YAML files (20 total)
  - 5 - deployment file
  - 5 - service file
  - 5 - configmap file
  - 5 - secret file (Do not commit this file to GitHub, just show it in your video)
- 5 - No AWS keys exposed anywhere
- 10 - Movie website is able to be viewed in a browser (via localhost)
- 10 - Self healing element (pod can be deleted and is automatically reconstructed)
- 5 - Three replicates
- 5 - Video demonstration of the cluster

Submission

Submit this assignment to your lab 4 repo on GitHub before the due date.

Example Files

For these files, there are overlaps in naming keys/values/both that are intentional. For example, the secret file the deployment needs must have the same name value in both files. So be aware that keys and values are intentionally assigned and, if a change happens in one file, one must check other files to ensure that everything stays linked.

Example Deployment YAML File

This file specifies what docker image will be deployed, as well as environment variables to be pulled (from secrets or configmaps)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: example-deployment
  labels:
    app: example-app
    // these labels can have whatever key-value pair you want, as long
    // as when you "matchLabels", you're matching to the same key-value pair
spec:
  replicas: 5 // will create 5 pods
  selector:
    matchLabels:
      app: example-app // matches labels above
  template:
    metadata:
      labels:
        app: example-app
    spec:
      containers:
      - name: example
        image: example-profile/example-image:latest
        ports:
          - containerPort: 80
        env:
        - name: FAVORITE_COLOR // key to be used when grabbing it in python
          valueFrom:
              secretKeyRef:
                name: example-secret
                key: favorite_color // key in secret YAML file
        - name: BUCKET
            valueFrom:
              configMapKeyRef:
                name: example-configmap
                key: bucket
```

Example Service YAML File

This file will specify the services needed for the app. In this lab, it'll be port numbers.

```
apiVersion: v1
kind: Service
metadata:
  name: example-service
  namespace: default
spec:
  type: NodePort
  selector:
    app: example-app // matches labels in different files
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30001
```

Example ConfigMap YAML File

This file specifies values that are not necessary to keep hidden. Essentially, it's environment variables that aren't sensitive.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: example-configmap
data:
  bucket: "example-bucket" // S3 bucket
  // Ports
  port: "80"
  target_port: "80"
  node_port: "30001"
```

Example Secret YAML File

This file specifies values that are necessary to keep hidden. Essentially, it's environment variables that are sensitive.

```
apiVersion: v1
kind: Secret
metadata:
  name: example-secret
  namespace: default
data: // note that these values are base64 encoded
  some_access_key: QUrJQTNMMzMzWknZVVU2RE9UNTQ=
  favorite_color: cmVk // yes, this is "red" base64 encoded, you can check
```